PROGRAM - 1

AIM:

To write a program and its algorithm to compute the diameter, area and perimeter of a circle using separate functions.Given the distance between the two points (x1,y1) and (x2,y2) is D=$\sqrt{(x2 - x1)^2 + (y2 - y1)^2}$.
Using concepts such as typecasting ,and symbolic constants.

ALGORITHM:

Step 1: Start
Step 2: Define PI = 3.14159
Step 3: Call Diameter()
Step 4: Stop

Diameter():
Step 1:Start
Step 2:Read coordinates x1,x2,y1,y2.
Step 3:Calculate diameter
Step 4:Display diameter
Step 5:Radius = (diameter)/2
Step 6:Call Area(radius)
Step 7:Call perimeter(radius)
Step 8:Display area and perimeter
Step 9:Stop.

Area(radius):
Step 1:Start
Step 2:Area=PI*r*r
Step 3:Return area
Step 4:Stop

Perimeter(radius):
Step 1:Start
Step 2:Perimeter=2*PI*r
Step 3:Return perimeter
Step 4:Stop

RESULT:

The diameter,area,perimeter of circle has been calculated successfully.
Diameter of circle is: 1.41
Area of circle is: 0.39
Perimeter of the circle is: 4.44.

PROGRAM:

```
#include <stdio.h>
#include <math.h>
//using symbolic constants
#define PI 3.14159

// Function Prototypes
double calculateDiameter(double x1, double y1, double x2, double y2);
double calculateArea(double radius);
double calculatePerimeter(double radius);

void main()
{
   double x1, y1, x2, y2;
   double diameter, area, perimeter, radius;

   printf("Enter coordinates of point 1 (x1 y1): ");
   scanf("%lf %lf", &x1, &y1);

   printf("Enter coordinates of point 2 (x2 y2): ");
   scanf("%lf %lf", &x2, &y2);

   diameter = calculateDiameter(x1, y1, x2, y2);
   radius = diameter / 2.0;

   // Function calls
   diameter = calculateDiameter(x1, y1, x2, y2);
   area = calculateArea(radius);
   perimeter = calculatePerimeter(diameter);

   // Displaying results with Typecasting
   printf("\n--- Results ---\n");
   printf("Diameter: %.2f\n", (float)diameter);
   printf("Area:     %.2f\n", (float)area);
   printf("Perimeter:%.2f\n", (float)perimeter);
}

// Function to compute distance/diameter
double calculateDiameter(double x1, double y1, double x2, double y2)
 {
   return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
```

```
}

// Function to compute area
double calculateArea(double diameter)
 {
   double radius = diameter / 2.0;
   return PI * pow(radius, 2);
}

// Function to compute perimeter
double calculatePerimeter(double diameter)
 {
   double radius = diameter / 2.0;
   return 2 * PI * radius;
}
```

OUTPUT:

```
Enter coordinates of point 1 (x1 y1): 5,
4
Enter coordinates of point 2 (x2 y2): 4,
3

--- Results ---
Diameter: 1.41
Area:     0.39
Perimeter:4.44
```

PROGRAM 2:

AIM:

To read marks of three subjects out of 100 from the user and to use separate functions to:

a. Find the largest among the numbers using nested if.
b. Find smallest among the numbers using if else if.
c. Calculate the average marks of three subjects. Use conditional operator to assign value 1 to a variable V if calculated average is equal to or greater than 50, and value 0 if otherwise.

ALGORITHM:

Step 1**:** Start.

Step 2**:** Call `ProcessMarks()`

Step 3**:** Stop.

Function ProcessMarks():

Step 1**:** Start.

Step 2**:** Read the three subject marks (m1,m2,m3).

Step 3**:** Call `FindLargest(m1, m2, m3)` and display the result.

Step 4**:** Call `FindSmallest(m1, m2, m3)` and display the result.

Step 5**:** Call `CalculateAverage(m1, m2, m3)` and display the result.

Step 6**:** Stop.

Function FindLargest(a, b, c):

Step 1**:** Start.

Step 2**:** Use **nested if**:

- If a>b:
  - If a>c, return a.
  - Else, return c.
- Else:
  - If b>c, return b.
  - Else, return c.

Step 3**:** Stop.

<u>Function FindSmallest(a, b, c):</u>

Step 1**:** Start.

Step 2**:** Use **if-else if**:

- If a<b and a<c, return a.
- Else if b<c, return b.
- Else, return c.

Step 3**:** Stop.

<u>Function CalculateAverage(a, b, c):</u>

Step 1**:** Start.

Step 2**:** Avg=(a+b+c)/3.

Step 3**:** Use **conditional operator**: V=(Avg>=50)?1:0.

Step 4**:** Display Avg and V.

Step 5**:** Stop.

RESULT:

The marks of three subjects have been successfully read from the user and the largest, smallest, and average mark has been computed. The variable V is assigned a respective value.

Largest Mark: 44
Smallest Mark: 23
Average Mark: 32.67
Variable V: 0 (Pass=1, Fail=0)

PROGRAM:

```
#include <stdio.h>
// Function Prototypes
void ProcessMarks();
int FindLargest(int a, int b, int c);
int FindSmallest(int a, int b, int c);
void CalculateAverage(int a, int b, int c);

void main()
 {
   ProcessMarks();
}

void ProcessMarks()
{
   int m1, m2, m3;

   printf("Enter marks for three subjects (out of 100):\n");
   scanf("%d %d %d", &m1, &m2, &m3);

   printf("\n--- Results ---");
   printf("\nLargest Mark:  %d", FindLargest(m1, m2, m3));
   printf("\nSmallest Mark: %d", FindSmallest(m1, m2, m3));
   CalculateAverage(m1, m2, m3);
   printf("\n");
}

// a) Finding Largest using nested if
int FindLargest(int a, int b, int c)
 {
     if (a > b)
     {
        if (a > c) return a;
        else return c;
      }
     else
      {
         if (b > c) return b;
         else return c;
       }
```

```
}

// b) Finding Smallest using if-else if
int FindSmallest(int a, int b, int c)
 {
      if (a <= b && a <= c)
     {
        return a;
      }
      else if (b <= c)
     {
        return b;
     }
     else
     {
        return c;
      }
}

// c) Calculate Average and use Conditional Operator
void CalculateAverage(int a, int b, int c)
 {
    float avg;
    int V;

    // Typecasting sum to float for accurate average
    avg = (float)(a + b + c) / 3;

    // Conditional Operator
    V = (avg >= 50) ? 1 : 0;

    printf("\nAverage Mark:  %.2f", avg);
    printf("\nVariable V:    %d (Pass=1, Fail=0)", V);
}
```

OUTPUT:

```
Enter marks for three subjects (out of 100): 23,31,44
--- Results ---
Largest Mark:  44
Smallest Mark: 23
Average Mark:  32.67
Variable V:    0 (Pass=1, Fail=0)
```

PROGRAM - 3:

AIM:

To write the algorithm and program to evaluate the following functions to 0.0001% accuracy.

a. sinx = $x - x^3/3! + x^5/5! - x^7/7! + \ldots$
b. cosx = $1 - x^2/2! + x^4/4! - x^6/6! + \ldots$
c. Sum = $1+(1/2)^2 + (1/3)^3 + (1/4)^4 + \ldots$

ALGORITHM:

Step 1:Start
Step 2: Define Symbolic Constants PI = 3.14159 and ACCURACY = 0.000001
Step 3 :Call EvaluateAllSeries()
Step 4: Stop

Function EvaluateAllSeries():

Step 1: Start
Step 2: Read the angle in degrees
Step 3: Convert degrees to radians: rad = deg * (PI / 180.0)
Step 4: Call Computesin(rad)
Step 5: Call Computecos(rad)
Step 6; Call ComputeSumSeries()
Step 7: Display all results using typecasting
Step 8: Stop

Function Computesin(x):

Step 1: Start
Step 2: Initialize sum = x, term = x, i = 1
Step 3: Start do-while loop
Step 4: Calculate next term: term = -term * x * x / ((2*i)* (2*i+1))
Step 5: Add term to sum
Step 6: Increment i by 1
Step 7: If term < 0 then check = -term, else check = term
Step 8: Repeat loop while check > ACCURACY
Step 9: Return sum
Step 10 Stop

Function Computecos(x):

Step 1: Start
Step 2: Initialize sum = 1.0, term = 1.0, i = 1
Step 3: Start do-while loop
Step 4 :Calculate next term: term = -term * x * x / ((2*i-1) * (2*i))
Step 5 :Add term to sum
Step 6: Increment i by 1
Step 7: If term < 0 then check = -term, else check = term
Step 8: Repeat loop while check > ACCURACY
Step 9: Return sum
Step 10: Stop

Function ComputeSumSeries():

Step 1: Start
Step 2 :Initialize sum = 0.0, i = 1
Step 3: Start do-while loop
Step 4 :Calculate term = 1.0 / pow(i, i)
Step 5 :Add term to sum
Step 6 :Increment i by 1
Step 7: Repeat loop while term > ACCURACY
Step 8: Return sum
Step 9: Stop

RESULT:

The program and algorithm of series expansion of a given sum function, sinx and cosx (as per taylor series) has been accurately evaluated to 0.0001% accuracy.

For x=30 degree:
sin(30.00) = 0.500000
cos(30.00) = 0.866026
SUM Series = 1.291286

PROGRAM:

```
#include <stdio.h>
#include <math.h>

// Symbolic Constants
#define PI 3.14159
#define ACCURACY 0.000001

// Function Prototypes
void EvaluateAllSeries();
double ComputeSin(double x);
double ComputeCos(double x);
double ComputeSumSeries();

void main()
 {
    EvaluateAllSeries();
}

void EvaluateAllSeries()
{
    double deg, rad;
    double sVal, cVal, sumVal;

    printf("Enter the angle in degrees: ");
    scanf("%lf", &deg);

    // Typecasting for calculation
    rad = deg * ((float)PI / 180.0);

    sVal = ComputeSin(rad);
    cVal = ComputeCos(rad);
    sumVal = ComputeSumSeries();

    printf("\n--- Results ---");
    printf("\nsin(%.2f) = %.6f", (float)deg, (float)sVal);
    printf("\ncos(%.2f) = %.6f", (float)deg, (float)cVal);
```

```
    printf("\nSUM Series = %.6f\n", (float)sumVal);
}

double ComputeSin(double x)
{
    double sum = x, term = x, check;
    int i = 1;
    do
{
        term = -term * x * x / ((2 * i) * (2 * i + 1));
        sum = sum + term;
        i++;

        if (term < 0)
       {
            check = -term;
        }
        else
       {
            check = term;
        }

    }
    while (check > ACCURACY);
    return sum;
}

double ComputeCos(double x)
 {
    double sum = 1.0, term = 1.0, check;
    int i = 1;
    do
    {
        term = -term * x * x / ((2 * i - 1) * (2 * i));
        sum = sum + term;
        i++;

        if (term < 0)
```

```
      {
          check = -term;
       }
       else
       {
          check = term;
       }

   }
   while (check > ACCURACY);
   return sum;
}

double ComputeSumSeries()
 {
   double sum = 0.0, term;
   int i = 1;
   do
  {
       term = 1.0 / pow((double)i, (double)i);
       sum = sum + term;
       i++;
   }
   while (term > ACCURACY);
   return sum;
}
```

OUTPUT:

```
Enter the angle in degrees: 30

--- Results ---
sin(30.00) = 0.500000
cos(30.00) = 0.866026
SUM Series = 1.291286
```

PROGRAM - 4:

AIM:

To write the algorithm and program to read a matrix and use function to find its saddle point and the second largest element without sorting.

ALGORITHM:

step 1: start
step 2: read row size m and column size n
step 3: read elements into matrix a[m][n]
step 4: call findSaddlePoints(a, m, n)
step 5: call findSecondLargest(a, m, n)
step 6: stop

function findSaddlePoints(a, m, n) :

step 1: start
step 2: start loop i from 0 to m-1
step 3: assume min_row is a[i][0] and col_ind is 0
step 4: find the smallest element in row i and store its column index in col_ind
step 5: assume this element is a saddle point
step 6: start loop k from 0 to m-1
step 7: if any element in column col_ind is greater than a[i][col_ind], then it is not a saddle point
step 8: if it remains a saddle point, display the value and its position
step 9: stop

function findSecondLargest(a, m, n):

step 1: start
step 2: initialize large = -99999 and second_large = -99999
step 3: start nested loops to traverse every element in the matrix
step 4: if current element > large, set second_large = large and large = current element
step 5: else if current element > second_large and current element != large, set second_large = current element
step 6: display second_large
step 7: stop

RESULT:

The algorithm and program have been successfully written using functions and the saddle point and second largest element for a given matrix has been computed.

For a 3 by 3 matrix:

enter rows and columns: 3
3
enter matrix elements:3,1,3,3,2,4,0,6,5
no saddle points found.
second largest element: 5

PROGRAM:

```
#include <stdio.h>
void findSaddlePoints(int a[10][10], int m, int n)
 {
    int i, j, k, min_row, col_ind, is_saddle;
    int found = 0;

    for (i = 0; i < m; i++)
   {
        min_row = a[i][0];
        col_ind = 0;
        for (j = 1; j < n; j++)
       {
            if (a[i][j] < min_row)
            {
                min_row = a[i][j];
                col_ind = j;
            }
        }

        is_saddle = 1;
        for (k = 0; k < m; k++)
        {
            if (a[k][col_ind] > min_row)
         {
                is_saddle = 0;
                break;
           }
        }

        if (is_saddle)
        {
            printf("\nsaddle point found: % d at [%d][%d]", min_row, i, col_ind);
            found = 1;
        }
    }
```

```
    if (!found) printf("\nno saddle points found.");
}

void findSecondLargest(int a[10][10], int m, int n)
{
    int i, j;
    int large = -32768;
    int second = -32768;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (a[i][j] > large)
            {
                second = large;
                large = a[i][j];
            }
             else if (a[i][j] > second && a[i][j] != large)
            {
                second = a[i][j];
            }
        }
    }

    if (second == -32768)
    {
        printf("\no second largest element found.");
    } else
    {
        printf("\nsecond largest element: %d", second);
    }
}

void main()
{
    int a[10][10], m, n, i, j;
```

```
    printf("enter rows and columns: ");
    scanf("%d %d", &m, &n);

    printf("enter matrix elements:\n");
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    findSaddlePoints(a, m, n);
    findSecondLargest(a, m, n);
    printf("\n");
}
```

OUTPUT:

```
enter rows and columns: 3
3
enter matrix elements:
3
3
0
1
2
6
0
6
5

no saddle points found.
second largest element: 5
```

PROGRAM - 5:

AIM:

To define a structure named student with roll number,name and marks as members and to sort the structure according to:

a. Alphabetical list of name,roll number,marks.
b. List sorted on roll number.
c. List sorted on marks.

ALGORITHM:

step 1: start
step 2: define a structure named student with roll_no, name, and marks
step 3: read the number of students n
step 4: read details for n students into an array of structures
step 5: call sortByName() and display the alphabetical list
step 6: call sortByRollNo() and display the list sorted by roll numbers
step 7: call sortByMarks() and display the rank-wise list (descending order)
step 8: stop

function sortByName(s, n):

step 1: start
step 2: use nested loops to compare s[i].name and s[j].name using strcmp
step 3: if s[i].name > s[j].name, swap the entire structure elements
step 4: stop

function sortByRollNo(s, n):

step 1: start
step 2: use nested loops to compare s[i].roll_no and s[j].roll_no
step 3: if s[i].roll_no > s[j].roll_no, swap the entire structure elements
step 4: stop

function sortByMarks(s, n):

step 1: start
step 2: use nested loops to compare s[i].marks and s[j].marks
step 3: if s[i].marks < s[j].marks (for descending order), swap the entire structure elements
step 4: stop

RESULT:

A structure named student has been successfully created and sorted based on alphabetical order,roll number and marks.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct student
 {
    int roll_no;
    char name[50];
    float marks;
};

void display(struct student s[ ], int n)
{
    printf("\n%-10s %-20s %-10s\n", "roll no", "name", "marks");
    printf("\n");
    for (int i = 0; i < n; i++)
   {
        printf("%-10d %-20s %-10.2f\n", s[i].roll_no, s[i].name, s[i].marks);
    }
}

void main()
{
    struct student s[100], temp;
    int n, i, j;

    printf("enter number of students: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nstudent %d details:\n", i + 1);
        printf("roll no: "); scanf("%d", &s[i].roll_no);
        printf("name: "); scanf("%s", s[i].name);
```

```
    printf("marks: "); scanf("%f", &s[i].marks);
}

// (a) alphabetical list
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (strcmp(s[i].name, s[j].name) > 0)
       {
            temp = s[i]; s[i] = s[j]; s[j] = temp;
        }
    }
}
printf("\n(a) alphabetical list of names:");
display(s, n);

// (b) sorted on roll numbers
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (s[i].roll_no > s[j].roll_no)
        {
            temp = s[i]; s[i] = s[j]; s[j] = temp;
        }
    }
}
printf("\n(b) list sorted on roll numbers:");
display(s, n);

// (c) rank-wise list (marks descending)
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (s[i].marks < s[j].marks)
       {
```

```
            temp = s[i]; s[i] = s[j]; s[j] = temp;
        }
    }
  }
  printf("\n(c) rank-wise list (sorted on marks):");
  display(s, n);
}
```

OUTPUT:

```
enter number of students: 3
student 1 details:
roll no: 1
name: NANDA
marks: 34

student 2 details:
roll no: 2
name: SANA
marks: 23

student 3 details:
roll no: 3
name: SHEEBA
marks: 45

(a) alphabetical list of names:
roll no    name            marks

1          NANDA            34.00
2          SANA             23.00
3          SHEEBA           45.00

(b) list sorted on roll numbers:
roll no    name            marks

1          NANDA            34.00
2          SANA             23.00
3          SHEEBA           45.00

(c) rank-wise list (sorted on marks):
roll no    name            marks
```

3	SHEEBA	45.00
1	NANDA	34.00
2	SANA	23.00